Running Your Own AI

D-Lab AI Pulse Workshop

Materials: `https://github.com/dlab-berkeley/AI_Pulse`
Feedback form: `https://forms.gle/x8KfejgCViE7t6iQA`

Why Run Your Own LLM?

Why you might want to:

- **Privacy:** Data never leaves your machine
- **Cost:** No API fees, no subscriptions
- **Customization:** Fine-tune to your needs

Why you might not:

- **Quality:** Smaller models, less capable
- **Setup:** More technical than a chat interface
- **Hardware:** Needs decent RAM/GPU for larger models

Roadmap

Today's application: using AI to help write a physics paper.

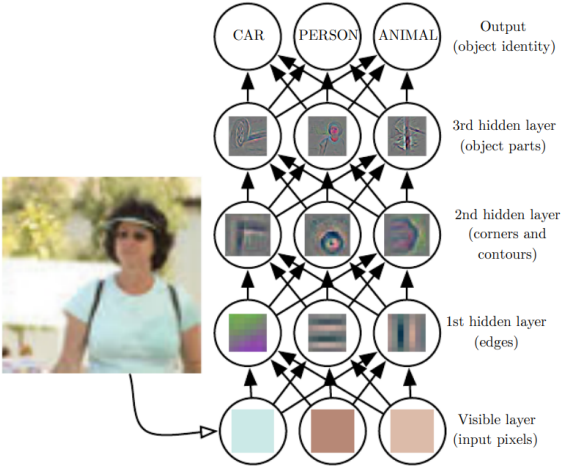
Not every task needs a frontier model. Smaller models have their own advantages.

1. **Run it:** Ollama
2. **Fit your paper in it:** context windows & quantization
3. **Make it yours:** LoRA fine-tuning on NRP
4. **Share it:** Hugging Face deployment

But first: 3 slides on how LLMs work, so that fine-tuning makes sense when we get there. Bear with me.

What is an LLM?

- LLMs are a type of **neural network**.
- Neural networks are made of **layers**.
- Learning from raw data (pixels) is very hard.
- Each layer models increasingly **abstract features**: edges, contours, object parts, identity.
- **Parameters:** the total count of numbers across all layers. More layers, more parameters.



But What Did It Actually Learn?

- These models learn **statistical patterns**, not **concepts**.
- More layers means more complex features: from edges to shapes to objects, from words to phrases to meaning.
- Adding layers changes how the network **thinks**, not what it knows.
- Today: **fine-tuning** (changing how it thinks). Not RAG (changing what it knows).
- This is why LLMs **hallucinate**: they produce statistically plausible text, not verified facts.

Ribeiro et al. (2016):
A husky/wolf classifier was detecting **snow in the background**, not the animal.

Fig. 11, arXiv:1602.04938

From Images to Language

A **Large Language Model (LLM)** predicts the next word based on patterns learned from massive text data. Not a database, not a search engine: a **pattern completer**.

"The cat sat on the _____"

`mat` (high probability) `quantum` (low probability)

Model	Parameters	File Size	RAM Needed
Llama 3.2 1B	1 billion	∼1.3 GB	∼2 GB
Llama 3.2 3B	3 billion	∼2.0 GB	∼4 GB
Llama 3.1 8B	8 billion	∼4.7 GB	∼8 GB
Llama 3.1 70B	70 billion	∼40 GB	∼48 GB

It's Just a File

You can download an LLM and run it on your laptop: no internet, no account, no subscription.

Ollama is a free, open-source app that downloads and runs LLMs on your machine.

Install:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows: Download from ollama.com

Supports models from Meta (Llama), Google (Gemma 4), Alibaba (Qwen 3), Microsoft (Phi-4), and more.

What Can Ollama Do?

It works like a browser LLM, but it can also be customized:

- **Cloud models:** some models are too large for your hardware. Ollama can run them on remote servers and send the results back to you.
- **Web search:** by default, models only know what they learned during training. Web search lets them look up current information before answering.
- **Context length:** controls how much text the model can "see" at once. A longer context lets you paste in entire papers, but uses more RAM.
- **Network access:** expose Ollama to your local network so other devices can connect to the same model. One powerful machine runs the model, everyone in the lab uses it.

AI for Writing

AI as a writing companion, not a ghost writer. Three reasons not to let AI write for you:

- It hallucinates: plausible-sounding text that may be wrong.
- It has a distinctive style: readers (and reviewers) can tell.
- It dulls your thinking (paper) and impairs skill formation (paper).

Instead, use it as a ping-pong partner, paragraph by paragraph:

- Have a few ideas but don't know how to connect them
- Want to avoid a repetitive word
- Want to reference a particular number, add a figure, or cite something
- Help review for consistency

Demo: Writing with AI

USER

I have these three ideas for a paragraph:

1. The Hamiltonian is symmetric but the ground state is not
2. This is called spontaneous symmetry breaking
3. It explains why phase transitions happen

Write a paragraph connecting these ideas. About 80 words, formal physics style.

ASSISTANT

"Although the governing Hamiltonian respects the full symmetry of the system, the ground state spontaneously selects a preferred configuration, breaking this symmetry. This phenomenon, known as spontaneous symmetry breaking, provides a unifying framework for understanding phase transitions in condensed matter systems. . ."

Demo: Ping-Pong

Now refine. You could say:

- "Be less formal"
- "This is too concise, expand the third point"
- "Avoid the word 'fundamental'"
- "Start with the phase transition, not the Hamiltonian"
- "Split this into two sentences"

ASSISTANT

"Phase transitions in condensed matter arise from a striking asymmetry: while the Hamiltonian treats all configurations equally, the ground state does not. The system spontaneously selects a preferred direction, breaking the underlying symmetry..."

Problem: It Forgot My Introduction

Paragraph-by-paragraph assistance works great with small models that fit on virtually any modern laptop (e.g., Llama 3.2 3B, just 2 GB).

But what if you want the model to reference something you wrote earlier in your paper? It can't. The model can only see ∼**3 pages** of text (4,096 tokens by default).

- It has no memory of your paper
- Only what's in the current conversation window
- Earlier context silently drops off

Can we fix this?

What Is a Context Window?

A token is the fundamental unit of information for LLMs (roughly 3/4 of a word).

The context window is the amount of text the model can "see" at once. Everything you send + everything it replies must fit in the window. Tradeoff: bigger context = more RAM.

Context Size	**Practical Use**	**RAM (3B model)**
4K (default)	Short conversations	∼3 GB
16K	A few pages of a paper	∼5 GB
32K	A full paper	∼7 GB
128K	Multiple papers	∼16 GB

Rule of thumb: 16K–32K is practical for most laptops.

Quantization: Making Models Fit

Full-precision models are huge: a 7B model is ∼14 GB. Quantization rounds the numbers to use fewer bits, like JPEG compression for AI. Smaller file, slight quality loss.

The RAM you save can be used for a larger context window.

Level	**Bits/Param**	**Quality**
FP16 (full)	16 bits	Original
Q8	8 bits	Near-original
Q4	4 bits	Good balance
Q2	2 bits	Very compressed, noticeable loss

This is why a 3B model fits in 2 GB instead of 6 GB. Models on Ollama's library come pre-quantized (usually Q4). No action needed.

Demo: Full Paper in Context

With a larger context window, you can paste an entire paper into the conversation and ask questions about it:

- "Summarize the main argument in 2 sentences"
- "Is the notation in Section 3 consistent with Section 2?"
- "Does the conclusion follow from the results?"

The model gives substantive answers referencing specific sections. It can proofread and check your paper for consistency.

Demo: paste `sample_paper.txt` into Ollama with 32K context

What's Still Missing

But ask it to write like your field, and you get generic text. "Magnetic frustration is a fascinating phenomenon..." sounds like a blog post, not a physics paper.

It doesn't know:

- What notation is standard in your subfield
- What a typical abstract sounds like
- What methods or terminology reviewers expect

It's helpful, but not specialized. What if we could teach it?

What Is Fine-Tuning?

Remember: adding layers changes how the network thinks, not what it knows.

Fine-tuning adds small new layers to a pre-trained model and trains them on your data. The original model stays frozen. The new layers learn to adjust how it thinks: writing style, terminology, conventions.

- Pre-training: done by Meta, Google, etc. Costs millions. Trains on the entire internet.
- Fine-tuning: done by you. Costs hours. Trains on your field's papers.

In Session 6, we saw Chemma: a 7B model fine-tuned on 1.28M chemistry Q&A pairs that outperformed GPT-4 at optimizing a novel reaction (Zhang et al., 2025).

The result: a model that thinks like a domain expert, not a generalist.

LoRA: The Trick That Makes It Possible

Full Fine-Tuning

Update **all** 3 billion parameters

- Needs huge GPU
- Tons of data required
- Produces a full copy of the model

LoRA (Low-Rank Adaptation)

Freeze original, add **small** trainable layers

- Train ∼1–10M new parameters
- Adapter file: ∼50–100 MB
- Fits on a single GPU

QLoRA: Same idea + quantize the base model → even less memory.

Analogy: adding sticky notes on a textbook.

Training Data: arXiv Papers

Goal: Teach the model the conventions of a physics subfield — in our case, `cond-mat.str-el` (strongly correlated electrons).

Source: 100 recent papers (Oct 2025 – Mar 2026) from arXiv. Full LaTeX source, not just abstracts.

Turning 100 papers into 2,282 training examples. From each paper we extract four kinds of prompt $\rightarrow$ completion pairs:

- *Title* $\rightarrow$ *abstract* — what should this paper emphasize?
- *Intro paragraph* $\rightarrow$ *continuation* — flow and coherence.
- *Section heading* + *context* $\rightarrow$ *section body* — structure and argument.
- *Paper body* $\rightarrow$ *abstract* — summarization in the field's voice.

One paper $\approx$ 25 examples. 100 papers $\rightarrow$ 2,282 train + 254 validation.

The model doesn't need a million papers — LoRA only adjusts a few million parameters. A few thousand high-quality examples is plenty.

Accessing NRP

The National Research Platform (NRP) is a free, NSF-funded GPU compute platform shared across US universities, including Berkeley.

Get access: Create a portal account with CalNet, then request a namespace in the Nautilus Support room on Matrix. Approval typically takes hours.

What you get: A JupyterHub interface at `jupyterhub-west.nrp-nautilus.io` with GPUs (A100, H100, RTX-series) and a "Deep Learning" image that ships with PyTorch + transformers preinstalled.

Warning — Pod Reclamation: NRP is a shared cluster. Your JupyterHub pod can be **killed without warning** if a higher-priority job needs the GPU. Our training took 3 attempts before finishing!

Protect yourself: save checkpoints frequently and back up outputs immediately.

Onboarding details

Training a LoRA Adapter

The Hugging Face ecosystem handles the heavy lifting. At a high level:

Load a pretrained model in 4-bit → **attach** a small LoRA adapter → **train** on your data → **save** the adapter (∼100 MB).

That's ∼50 lines of Python. You don't need to understand every hyperparameter to get started.

Tutorials to learn from:

- Hugging Face PEFT docs — the canonical reference
- Maxime Labonne's LLM Course — free, hands-on notebooks
- Philipp Schmid's QLoRA walkthrough — single blog post, very clear

Where it runs: Any CUDA GPU with ∼12 GB VRAM — NRP, Savio, Colab, or a laptop. Our notebook is in the workshop repo.

Where Will You Run It? & Prior Art in Science

Where will you run it? Pick a base model that fits the hardware.

- **Laptop** ($\leq$16 GB RAM): 1B–3B parameters. Qwen 2.5 3B, Llama 3.2 3B, Gemma 3 4B.
- **Workstation or one GPU:** 7B–8B. Llama 3.1 8B, Qwen 2.5 7B.
- **Shared cluster** (NRP, Savio): 13B–70B when you need the extra capability.

Successful scientific fine-tunes to learn from:

- **Meditron-70B** (EPFL, 2023): Llama 2 fine-tuned on PubMed + medical guidelines. Matches GPT-3.5 on clinical benchmarks.
- **BioGPT** (Microsoft, 2022): GPT-2 fine-tuned on 15M PubMed abstracts. Strong at biomedical relation extraction.
- **BloombergGPT** (2023): 50B-parameter model trained on 363B tokens of financial text. Domain outperforms general LLMs on finance tasks.
- **Chemma** (Zhang et al., 2025): 7B fine-tuned on 1.28M chemistry Q&A pairs. Beat GPT-4 at reaction optimization.

Demo 3a: Base Model vs. Fine-Tuned Model

Prompt: "Write an abstract for a paper titled: *Pressure-induced superconductivity in the kagome metal CsV_3Sb_5*."

Base Model

"The discovery of pressure-induced superconductivity in the kagome metal CsV_3Sb_5 represents a significant advance in the field of unconventional superconductors. This study investigates the effect of hydrostatic pressure on the electronic and magnetic properties. . . "

Review-paper opener; vague "represents a significant advance."

Fine-Tuned Model

"The kagome compound CsV_3Sb_5 is reported to exhibit superconductivity at 12 K under high pressure. Here we report that this originates from a direct band crossing of the V-$d_{x^2-y^2}$ orbital, supported by first-principles calculations. . . "

Specific mechanism, orbital notation, "Here we report. . . "

Demo 3b: Writing in the Field's Voice

Prompt: "Continue this paragraph from a condensed matter physics paper: *The interplay between strong electron correlations and spin-orbit coupling has emerged as a central theme in the study of 4d and 5d transition metal oxides. In these systems,...*"

Base Model

"...the unique combination of these effects leads to fascinating phenomena such as unconventional superconductivity, topological insulators, and quantum spin liquids. Understanding these interactions is crucial for unraveling..."

Lists buzzwords; reads like a review.

Fine-Tuned Model

"...the strong SOC induces nontrivial Berry curvature for both itinerant electrons and localized moments, leading to a complex energy spectrum and enhanced many-body effects [citation]. While much progress has been made..., the full phase diagram ...remains largely uncharted."

Specific mechanism, `[citation]` placeholder, hedged claim.

Caveat: it learned *statistical patterns*, not ground truth — always verify.

From NRP to the World

You trained a model. It lives on an NRP node. Now what?

Hugging Face = GitHub for AI models. Upload the adapter (∼130 MB) once, anyone can pull it down.

```
hf auth login --token $HF_TOKEN
hf upload brunosmaniotto/condmat-qwen25-qlora-fullpapers \
    ./physics-adapter-fullpapers . --repo-type=model
```

Our adapter is live: `hf.co/brunosmaniotto/condmat-qwen25-qlora-fullpapers`

A good model card includes:

- Intended use and limitations
- Load snippet (so people can actually run it)
- Training data, hyperparameters, hardware
- License of the base model you built on

Deployment Options

Option	Cost	Best For
Ollama on your laptop	Free	Personal use, privacy
Google Colab	Free (Pro via Berkeley)	Interactive, faster GPU
Kaggle Notebooks	Free (30 hrs/wk GPU)	Reproducible workflows
Hugging Face Spaces	Free (ZeroGPU)	Shareable demo apps

Key: Berkeley provides **Colab Pro** for free — gives you T4/V100 GPUs and longer runtimes.

For most researchers: start with Ollama locally, use Colab for collaboration.

When to Fine-Tune vs. When to Just Prompt

A useful distinction: does your task need complex instructions, or complex thinking?

- Complex instructions (prompt engineering): "write this paragraph in formal style, avoid passive voice, use these three terms." The model already knows how. You just need to tell it what you want.
- Complex thinking (fine-tuning): "write like a condensed matter physicist." The model doesn't know how. You need to change how it thinks.

	Prompt Engineering	**Fine-Tuning**
What it does	Steers a generalist with instructions	Changes how the model thinks
Example	"Rewrite this more formally"	Training on 100 full arXiv papers
Effort	Zero setup	Hours of training, needs GPU
Data stays	With the provider (unless local)	Local, the adapter is yours

RAG vs. Fine-Tuning: Information vs. Thinking

RAG (Retrieval-Augmented Generation)

Gives the model new information.

- Fetches relevant documents at query time
- Model reads them as context, then answers
- Great for: facts, policies, recent data, specific documents

Fine-Tuning (LoRA / QLoRA)

Changes how the model thinks.

- Modifies the model's weights permanently
- Model internalizes style, conventions, reasoning patterns
- Great for: tone, domain style, specialized writing, field conventions

Use RAG when you need the model to know specific things. Use fine-tuning when you need it to write or reason like a domain expert. They can also be combined.

Questions?

Appendix: How We Onboarded to NRP

1. **Create a portal account** at portal.nrp-nautilus.io using CalNet (CILogon).
2. **Request a namespace** by posting in the **Nautilus Support** room on Matrix (via the Element client). The admin team reviews requests directly.
3. **The message we sent:**

> *Hi, I'm a researcher at UC Berkeley (D-Lab). I'd like to request a new namespace for running AI/ML workshops — specifically for fine-tuning open-weight LLMs with LoRA as part of a teaching series. My portal account is* `brunosmaniotto@berkeley.edu`*. Could I be promoted to namespace admin? Suggested namespace name:* `berkeley-dlab`*. Thank you!*

4. **Wait for approval.** Ours was approved within hours.
5. **Log in** at `jupyterhub-west.nrp-nautilus.io` with CalNet, launch a server with the *NRP Deep Learning* image and a GPU.

Back to Accessing NRP